This compatibility match does not necessarily go in both directions if the two classes have different property lists. Let's add a property to Shirt:

```
class Hat {
  constructor(private size: number) {}
}

class Shirt {
  constructor(private size: number, private sleeves: string) {}
}
```

Now we've added a second property to shirt. Which of these is now legal?

```
let x: Hat = new Shirt()
let x: Shirt = new Hat()
```

The first line is still legal—Hat only has one property, and Shirt still shares it. But the second line is now a type error because Shirt has a property, sleeves, that is not contained by Hat.

The same basic idea of compatibility holds when you need to determine if two functions are type compatible with each other—meaning whether two functions, as first-class items in the system, can be used in the same place. For every parameter in the function type on the left side of the assignment, there must be a matching parameter (in order) on the right side. The right side can have extra parameters since passing extra parameters to a function and having them be ignored is not at all unusual in JavaScript.

In general, whether a parameter or property is optional or required does not make a difference as far as type compatibility is concerned.

If you are used to Java, or Elm, or some other strictly typed language, the TypeScript rules here may seem odd, and in a way, they are. TypeScript is more permissive than type systems in those languages because they can allow objects of unrelated types to be assigned. In fact, the TypeScript documentation discusses some edge cases where the compiler might allow code that turns out to be unsafe at run time. (We're not going to worry about those cases right here, but if you are curious, check them out on the TypeScript website.)[51]